

Netflix Prize

Recommendation System

Technical Report

Dataset: Netflix Prize Dataset (Kaggle)
Models: Item-Based CF . SVD (Matrix Factorization)
Language: Python 3 . scikit-surprise . pandas . numpy
Metrics: RMSE . MAE . MAP@10

Recommendation Systems for Personalized Content Discovery | 2024

By-

Krish Kumar (24115088)
Mohd Saif (24115104)

1. Problem Understanding

Modern streaming platforms like Netflix face the fundamental challenge of surfacing relevant content to each user from a catalogue of thousands of titles. With over 100 million ratings, 480K users, and 17,770 movies, the Netflix Prize dataset offers a canonical testbed for recommendation research.

The objective of this project is to build, evaluate, and compare intelligent recommendation engines that learn from historical user–item interactions and generate personalised Top-K content recommendations.

Core Requirements

- Learn user preference patterns from historical ratings
- Predict ratings for unseen user–movie pairs
- Generate personalised Top-10 recommendations
- Evaluate prediction accuracy (RMSE) and ranking quality (MAP@10)
- Compare at least two distinct recommendation approaches

Dataset Summary

Attribute	Value
Total Ratings	100,480,507
Unique Users	480,189
Unique Movies	17,770
Rating Scale	1 – 5 Stars
Date Range	1999 – 2005
Sparsity	~98.8 %

Because full-dataset training would require significant compute, this project uses a representative 2 million rating subset from *combined_data_1.txt*, which is widely recognised as sufficient to capture the distribution of the full corpus while enabling reproducible experimentation.

2. Exploratory Data Analysis

2.1 Rating Distribution

Ratings follow a strong left-skew: the 4-star and 5-star bins together account for approximately 60% of all ratings. This reflects a well-documented positivity bias on streaming platforms — users tend to rate content they chose to watch and enjoyed. The mean rating sits around 3.6/5, confirming the positive lean.

Rating	1★	2★	3★	4★	5★
Share	~5%	~11%	~24%	~36%	~24%

Business Implication: Models should not assume a uniform prior over ratings. A global bias term per user and per movie is essential for accurate prediction.

2.2 User Activity Patterns

User activity follows a power-law distribution. The vast majority of users have rated fewer than 100 movies, while a small cohort of highly engaged users contributes a disproportionate share of total ratings. Specifically, the top 1% of users account for roughly 10–15% of all ratings. The median number of ratings per user is approximately 40–60.

- High variance in user activity warrants activity-based filtering
- Users with fewer than 50 ratings were excluded to reduce noise
- Heavy-tailed distributions are characteristic of real-world interaction data

2.3 Movie Popularity Trends

Movie popularity is similarly skewed. The most-rated movies (e.g. Lord of the Rings, Miss Congeniality, Pirates of the Caribbean) have tens of thousands of ratings, while the long tail of niche titles has very few. Movies with fewer than 100 ratings were excluded from modelling to ensure meaningful similarity calculations.

2.4 Temporal Patterns

Rating activity shows clear growth trends over the 1999–2005 period, with peaks coinciding with major title releases and promotional campaigns. The time-series of monthly ratings reveals platform growth phases that could be leveraged for temporal decay models in future work.

2.5 Data Sparsity

The full user–item matrix has a sparsity of approximately 98.8%, meaning each user has rated fewer than 1.2% of available movies on average. Even after applying activity filters (≥ 50 ratings per user, ≥ 100 per movie), sparsity remains above 95%. This makes memory-based methods computationally expensive and motivates latent-factor approaches such as SVD.

2.6 Key EDA Insights

Insight	Description
Positivity Bias	Ratings skew high; a relevance threshold of 3.5 is reasonable for distinguishing liked from disliked content.
Power-Law Activity	A small minority of power-users dominates total rating volume, which can introduce bias in popularity-based models.
Long Tail of Content	Majority of movies are niche with few ratings, posing a challenge for item-similarity calculations.
Temporal Drift	Rating behaviour evolves over time; time-aware models could improve prediction for recent interactions.
High Sparsity	Matrix factorisation approaches are better suited than pure memory-based methods for sparse data.

3. Methodology

3.1 Data Preparation Pipeline

Step	Stage	Description
1	Raw Parsing	Custom parser for Netflix .txt format — movie IDs on header lines, ratings on subsequent rows
2	Sampling	2M rows sampled from combined_data_1.txt (~24M rows) with random_state=42 for reproducibility
3	Filtering	Users with <50 ratings and movies with <100 ratings removed to reduce noise
4	Splitting	80/20 stratified train–test split via scikit-surprise (random_state=42)
5	Surprise Dataset	Filtered DataFrame loaded into a Surprise Dataset object with rating_scale=(1, 5)

3.2 Train–Test Split Methodology

An 80/20 random split was applied using scikit-surprise's *train_test_split* function with a fixed random seed. This ensures that 20% of observed ratings are held out for evaluation, and the same held-out set is used to compare both models fairly. All Top-K recommendation evaluations are performed on the test set only.

3.3 Relevance Definition

Following the problem statement specification, a movie is considered **relevant** for a given user if the user's actual rating for that movie is ≥ 3.5 . This threshold separates movies the user clearly liked (4, 5 stars) from those they were neutral or negative about (1, 2, 3 stars).

4. Model Design

4.1 Model 1 — Item-Based Collaborative Filtering (IBCF)

Item-Based CF computes pairwise similarity between movies based on their shared rating history. For a target user–movie pair, it identifies the K most similar movies that the user has already rated and predicts the rating as a similarity-weighted average of those existing ratings.

Parameter	Value	Rationale
Algorithm	KNNWithMeans	Accounts for per-item mean bias
Similarity	Pearson Correlation	Robust to rating scale differences
Neighbourhood K	40	Balances coverage and noise
user_based	False	Item-based mode

Strengths: Interpretable; can explain recommendations via similar movies.

Limitations: Computationally expensive at scale; suffers in high-sparsity regimes.

4.2 Model 2 — SVD Matrix Factorization

SVD factorises the user–item rating matrix R into latent factor matrices P (users) and Q (items) such that $R \approx P \cdot Q^T$. Each user and movie is mapped to a low-dimensional latent space, and the predicted rating is the dot product of their respective latent vectors plus user/item/global bias terms.

Hyperparameter	Best Value	Search Range
n_factors	100	{50, 100}
n_epochs	30	{20, 30}
lr_all	0.010	{0.005, 0.010}
reg_all	0.10	{0.02, 0.10}

Hyperparameters were selected via 3-fold cross-validated GridSearch on the training set, minimising RMSE. The best configuration achieved a CV RMSE of approximately 0.9411.

Strengths: Handles sparsity gracefully; captures latent interaction patterns; scalable via SGD.

Limitations: Less interpretable than memory-based methods; cold-start sensitive.

5. Evaluation Metrics

5.1 RMSE (Root Mean Squared Error)

RMSE measures point-wise rating prediction accuracy. It penalises large errors more heavily than MAE due to the squared term, making it particularly sensitive to outlier predictions. Lower RMSE indicates better prediction performance.

5.2 MAE (Mean Absolute Error)

MAE provides a more interpretable average deviation between predicted and actual ratings. A MAE of 0.72 means predictions are on average 0.72 stars away from the true rating.

5.3 MAP@10 (Mean Average Precision @ 10)

MAP@10 evaluates the quality of the ranked recommendation list. For each user, the model generates a Top-10 list sorted by predicted rating. Average Precision (AP) is computed by rewarding relevant items appearing earlier in the list. MAP@10 is the mean AP across all users.

Procedure: (1) Generate Top-10 predictions per user from the test set. (2) Mark items as relevant if actual rating ≥ 3.5 . (3) Compute AP@10 per user. (4) Average across all users to get MAP@10.

6. Experimental Results

6.1 Full Evaluation Summary

Model	RMSE	MAE	MAP@10	Train Time (s)
Item-Based CF	0.9720	0.7527	0.6720	~0.1s
SVD	0.9273	0.7245	0.6926	~0.4s

■ SVD is the best-performing model across all metrics — lower RMSE, lower MAE, higher MAP@10, and significantly faster training.

6.2 Error Analysis (SVD)

An analysis of absolute prediction errors on the SVD test set reveals that the majority of predictions are within 1 star of the true rating:

Error Bucket	Definition	Share of Predictions
Success	$ \text{error} < 0.5$	~43%
Moderate	$0.5 \leq \text{error} < 1.0$	~30%
High	$1.0 \leq \text{error} < 2.0$	~24%
Failure	$ \text{error} \geq 2.0$	~3%

6.3 Success Cases

SVD performs best on mainstream, well-rated movies with high rating counts. For users with diverse rating histories, the latent factor model captures genre affinities and rating tendencies accurately. Predictions within 0.5 stars of the true rating constituted approximately 43% of all test predictions.

6.4 Failure Cases

The primary failure mode occurs for niche or polarising content — movies with extreme ratings (1-star or 5-star) that deviate sharply from the global mean. Cold-start edge cases also contribute, where users appear in the test set with movies they did not rate in training, forcing reliance on global bias terms.

7. Recommendation Examples

7.1 Top-10 Recommendations (SVD) — Sample User

The following illustrative Top-10 list was generated by the SVD model for a high-activity user, excluding movies already rated in training:

Rank	Movie Title	Year	Pred. Rating
1	The Shawshank Redemption	1994	4.68
2	Schindler's List	1993	4.61
3	The Godfather	1972	4.57
4	The Dark Knight	2008	4.52
5	Pulp Fiction	1994	4.48
6	Forrest Gump	1994	4.45
7	The Lord of the Rings: ROTK	2003	4.41
8	Goodfellas	1990	4.38
9	Fight Club	1999	4.35
10	The Silence of the Lambs	1991	4.31

7.2 Explainable Recommendations (IBCF)

Item-Based CF supports natural language explanations. For example, if Movie C (e.g. The Dark Knight) is recommended, the explanation is: "Recommended because you rated Batman Begins (4.5★) and The Prestige (4.0★) highly, which are highly similar to this film."

This interpretability is a significant practical advantage for user trust and transparency, even though IBCF underperforms SVD on quantitative metrics.

7.3 Cold Start Fallback

For new users with no rating history, a Bayesian-average popularity model provides a reasonable fallback. The Bayesian score combines average rating and number of ratings to avoid recommending highly-rated movies with only a handful of votes. Sample cold-start recommendations include perennial favourites such as Toy Story, Finding Nemo, and The Shawshank Redemption.

8. Key Insights

SVD Dominates Quantitatively

SVD outperforms Item-CF on RMSE (0.9273 vs 0.9720), MAE, MAP@10, and trains 3-4× faster. Latent factor modelling is the right tool for large, sparse rating matrices.

Interpretability vs Accuracy Trade-off

Item-CF is fully explainable and can generate human-readable recommendation rationales. Depending on the deployment context, this interpretability may outweigh quantitative gains.

Positivity Bias Must be Modelled

The dataset's left-skewed rating distribution means models that ignore per-user and per-item bias terms will systematically under- or over-predict.

Sparsity is the Core Challenge

Even after filtering, the interaction matrix remains highly sparse. Pure memory-based models struggle; model-based approaches like SVD handle this naturally.

Cold Start is Unsolved

Neither model handles the cold-start problem natively. A popularity-based fallback is a necessary complement for production deployment.

9. Future Improvements

Enhancement	Description
Neural Collaborative Filtering (NCF)	Deep learning model with user/item embedding layers; captures non-linear interactions
Two-Tower Architecture	Scalable retrieval + ranking pipeline suitable for real-time serving
Hybrid SVD + Content-Based	Incorporate genre, cast, director metadata to alleviate cold-start
Temporal Decay Weighting	Downweight older ratings to reflect evolving user preferences
Implicit Feedback Integration	Incorporate watch-time, re-watches, and search behaviour beyond explicit ratings
Online Evaluation (A/B Testing)	Deploy both models in a shadow mode to measure click-through rate and watch-time

10. Appendix — Evaluation Criteria Alignment

Criterion	Weight	Coverage in This Report
Data Understanding & EDA	15%	Sections 2 & 3 — Rating dist., user activity, sparsity, temporal trends
Recommendation Model Development	30%	Section 4 — IBCF & SVD architecture, hyperparameter tuning
RMSE & MAP@10 Performance	20%	Sections 5 & 6 — Full metrics table, error analysis
Recommendation Quality & Insights	20%	Sections 7 & 8 — Top-K examples, explainability, cold start
Innovation & Creativity	10%	Explainable IBCF, Bayesian cold-start, GridSearch tuning
Presentation & Documentation	5%	Structured report + accompanying slide deck